

Divide&Conquer

Task: 三大数相乘优化算法

Implementation

三数乘法采用最根本的：

$$(A * B) * C$$

Karatsuba

将整数拆分为两部分，减少乘法次数：

$$A = A_1 \cdot 10^m + A_0$$

时间复杂度为：

$$O(n^{\log_2 3})$$

Toom-3

将整数拆成三段：

$$A = a_0 + a_1x + a_2x^2$$

步骤：

1. 在 5 个点求值 $(0, 1, -1, 2, \infty)$
2. 递归计算
3. 插值恢复结果

相当于解一个线性方程组以求得最后每个系数的表达方式，最终将9次乘法变成5次。

时间复杂度为：

$$O(n^{\log_3 5})$$

FFT

这个利用了傅里叶变换，转化为卷积，时间复杂度可以做到：

$$O(n \log n)$$

但在小规模的时候，由于常数项和实现的开销较大，实际上会慢一些，在这里主要当做一个SOTA的参考价值，并没有研究其中的道理，可以作为test验证前两个算法是否正确。

Test results & Analysis

通过验证发现时间复杂度和最后time计时器得出的消耗时间基本一致，三个算法的结果都一致，详细见[3Num-Divide&Conquer.ipynb](#)

Difficulties & Solutions

Toom-3 插值复杂，容易写错，通过分步骤验证，并用小数据调试解决问题。

Task: 多项式乘法

Implementation

实现方法基本和大数乘法的分治算法一致，利用最基本的karatsuba算法加速多项式乘法，假设我们要计算两个多项式的乘积，且它们的阶数均为n（若不足则补零），则：

$$A \cdot B = (A_{high}B_{high})x^{2m} + (A_{high}B_{low} + A_{low}B_{high})x^m + (A_{low}B_{low})$$

$$Z_0 = A_{low} \cdot B_{low}$$

$$Z_2 = A_{high} \cdot B_{high}$$

$$Z_1 = (A_{low} + A_{high}) \cdot (B_{low} + B_{high}) - Z_0 - Z_2$$

最后合并：

$$D(x) = Z_2 \cdot x^{2m} + Z_1 \cdot x^m + Z_0$$

- 设置一个naive_multiply函数作为朴素的基础算法，作为多项式乘法的baseline；

```
def naive_multiply(A, B):
    n = len(A)
    m = len(B)
    result = [0] * (n + m - 1)

    for i in range(n):
        if A[i] == 0: continue
        for j in range(m):
            result[i + j] += A[i] * B[j]
    return result
```

- add_poly(A, B)/sub_poly(A, B)，实现多项式的加减法，完成中间项的计算。

```
def add_poly(A, B):
    result = [0] * max(len(A), len(B))
    for i in range(len(A)):
        result[i] += A[i]
    for i in range(len(B)):
        result[i] += B[i]
    return result

def sub_poly(A, B):
    result = [0] * max(len(A), len(B))
    for i in range(len(A)):
        result[i] += A[i]
    for i in range(len(B)):
        result[i] -= B[i]
    return result
```

- karatsuba(A, B)，实现具体的分治过程，接受两个系数列表，返回乘积系数列表，先补齐、再计算、最后切片组成result列表。

```

def karatsuba(A, B):
    n = max(len(A), len(B))
    if n <= THRESHOLD:
        return naive_multiply(A, B)

    if n % 2 != 0:
        n += 1
    A = A + [0] * (n - len(A))
    B = B + [0] * (n - len(B))

    m = n // 2

    A_low = A[:m]
    A_high = A[m:]

    B_low = B[:m]
    B_high = B[m:]

    z0 = karatsuba(A_low, B_low)
    z2 = karatsuba(A_high, B_high)

    sum_A = add_poly(A_low, A_high)
    sum_B = add_poly(B_low, B_high)
    z1_temp = karatsuba(sum_A, sum_B)

    z1 = sub_poly(z1_temp, z0)
    z1 = sub_poly(z1, z2)

    result = [0] * (2 * n - 1)

    for i in range(len(z0)):
        result[i] += z0[i]
    for i in range(len(z1)):
        result[i + m] += z1[i]
    for i in range(len(z2)):
        result[i + 2 * m] += z2[i]

    return result

```

- 最后写成输出表达式的形式;

```

def poly_to_string(coeffs):
    if not coeffs:
        return "0"
    terms = []
    for i, c in enumerate(coeffs):
        if c == 0:
            continue

        abs_c = abs(c)
        sign = "-" if c < 0 else "+"

        term_str = ""
        if i == 0:
            term_str = f"{abs_c}"

```

```
elif i == 1:
    if abs_c == 1:
        term_str = "x"
    else:
        term_str = f"{abs_c}x"
else:
    if abs_c == 1:
        term_str = f"x^{i}"
    else:
        term_str = f"{abs_c}x^{i}"

if i == 0:
    terms.append(f"{-abs_c}" if c < 0 else f"{abs_c}")
else:
    terms.append(f" - {term_str}" if c < 0 else f" + {term_str}")
if not terms:
    return "0"
return "".join(terms)
```

Test results & Analysis

- 系数范围-20~20
- 阈值设置32

随着n的增大，Karatsuba算法的优势逐渐显现。在n较小时，由于Python函数调用和列表切片的开销，分治算法的优势不明显，甚至略慢，因此设置阈值切换到朴素算法是必要的优化。

Performance Comparison: Naive vs Karatsuba				
Size	Naive(ms)	Karatsuba(ms)	Speedup	Status
64	0.5965	0.7377	0.81	x ✓
128	2.5456	1.5218	1.67	x ✓
512	44.1291	20.6831	2.13	x ✓
1024	200.1073	52.0124	3.85	x ✓
4096	N/A	450.2089	N/A	Skipped (too slow)

Difficulties & Solutions

从大数切换到多项式，即从正常的数字切换到列表做分治难以下手，即切片和多项式加减有点难操作，通过思考以及AI辅助编程完成代码的构建。

测试部分均由qwen3.5-Plus辅助完成。